

★ Member-only story

Understanding the semantics of ontology-based reasoning

From Property Graphs to OWL Inference Using a Hybrid Pipeline

4 min read · Feb 11, 2026



Akash Goyal

Follow



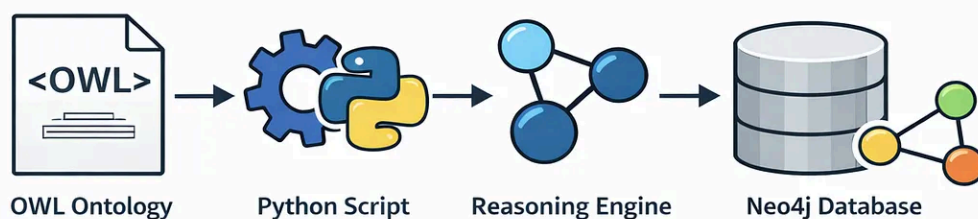
Listen



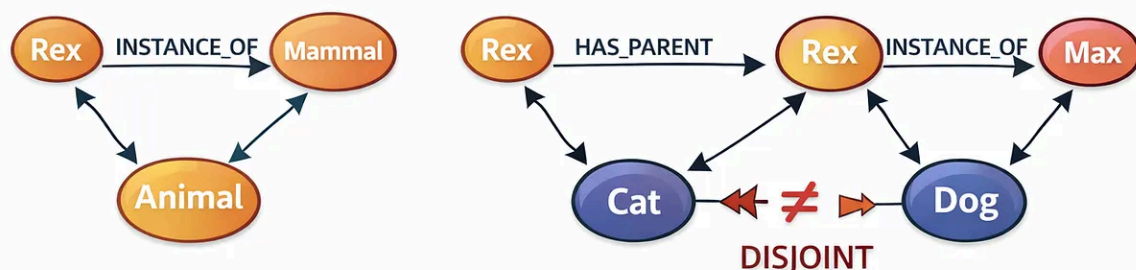
Share

... More

Ontology-Based Reasoning Pipeline



Examples of Inference



Modern knowledge graphs often sit at an interesting intersection.

On one side, we have **graph databases** like Neo4j that are excellent at traversal, analytics, and operational performance. They shine when we need fast queries, relationship exploration, and application serving.

On the other side, we have **semantic technologies** like **OWL ontologies** and reasoners such as Apache Jena or Owlready2 that excel at something Neo4j does not do natively: **formal logical inference**.

If you are not a medium member click here [Medium Friend Link](#)

This difference is subtle but fundamental.

Neo4j answers:

“What paths exist in my stored graph?”

OWL reasoners answer:

“What facts must logically be true, even if not stored?”

When building real knowledge systems, we often want both. This article walks through the concepts and then shows how to build a **fully automated hybrid pipeline** that combines them.

Understanding the conceptual difference

Let's start with the mental model.

A **property graph** (Neo4j's model) stores nodes and relationships explicitly. If an edge is not stored, it does not exist.

An **ontology reasoner** works differently. It interprets formal semantics and derives new truths automatically.

Suppose your ontology states:

- $\text{Dog} \subseteq \text{Mammal}$
- $\text{Mammal} \subseteq \text{Animal}$

- Rex is a Dog

A reasoner infers:

- Rex is a Mammal
- Rex is an Animal

These facts were never inserted. They are *logically implied*.

Neo4j will not add these automatically. You either query traversals or manually materialize them.

That's the core semantic gap.

The hybrid approach

Instead of forcing Neo4j to behave like a theorem prover, we separate responsibilities:

Neo4j handles:

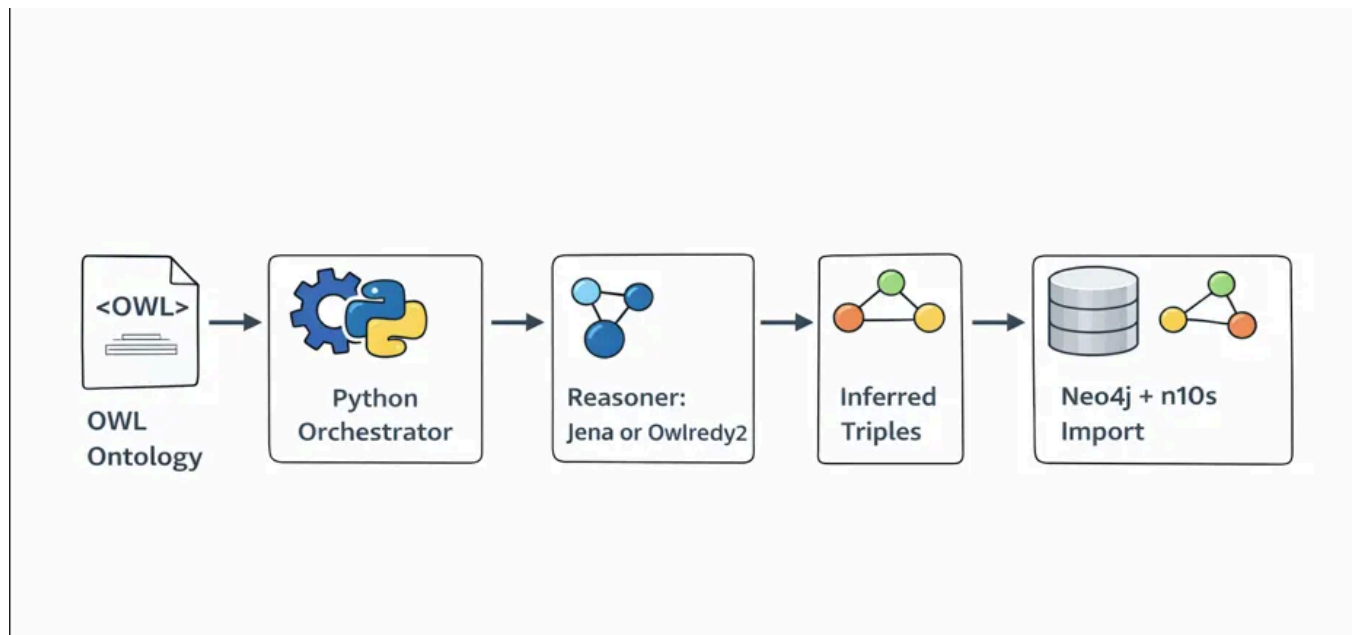
- storage
- queries
- graph analytics
- serving applications

The reasoner handles:

- subclass inference
- transitive properties
- disjoint class detection
- logical consistency

We periodically export the graph to RDF, run reasoning, and import the inferred facts back.

Conceptually, it looks like this:



Neo4j becomes faster and simpler because inference is precomputed.

What the data looks like

Imagine this small knowledge domain:

- Dog subclassOf Mammal
- Mammal subclassOf Animal
- hasParent is transitive
- Rex hasParent Buddy
- Buddy hasParent Max

Reasoning should infer:

- Rex is Animal
- Rex hasParent Max

The logical structure:

```
Dog --> Mammal --> Animal
Rex -->|instanceOf| Dog
Rex -->|hasParent| Buddy -->|hasParent| Max
```

The reasoner adds hidden edges:

```
Rex -->|instanceOf| Mammal
Rex -->|instanceOf| Animal
Rex -->|hasParent| Max
```

Implementing the pipeline

Now we move from theory to working code.

We will use:

- Neo4j
- Neo4j Neosemantics (n10s)
- Python
- Owlready2 (built-in HermiT reasoner)

This avoids heavy Java setup while still giving full OWL inference.

Step 1 — Start Neo4j with n10s

Install the n10s plugin in Neo4j and enable:

```
dbms.security.procedures.unrestricted=n10s.*
```

Initialize once:

```
CALL n10s.graphconfig.init({  
  handleVocabUris: "IGNORE"  
});
```

Step 2 — Your OWL file

Suppose you already have:

```
ontology.owl
```

It contains:

```
Dog ⊆ Mammal  
Mammal ⊆ Animal  
hasParent is Transitive
```

No individuals yet.

Step 3 — Automation script

Now we automate everything.

Create:

pipeline.py

Full working script

```
import sys
from owlready2 import *
from neo4j import GraphDatabase
NEO4J_URI = "bolt://localhost:7687"
USER = "neo4j"
PASS = "password"

def run_reasoner(owl_path, out_path):
    print("Running reasoning...")
    onto = get_ontology(owl_path).load()
    with onto:
        sync_reasoner()
    onto.save(file=out_path, format="rdxml")

def import_into_neo4j(rdf_file):
    driver = GraphDatabase.driver(NEO4J_URI, auth=(USER, PASS))
    with driver.session() as session:
        session.run("""
            CALL n10s.rdf.import.fetch(
```

Open in app ↗

≡ Medium



```
def materialize_rules():
    driver = GraphDatabase.driver(NEO4J_URI, auth=(USER, PASS))
    with driver.session() as session:
        session.run("""
            MATCH (i)-[:INSTANCE_OF]->(c)-[:SUBCLASS_OF*]->(sup)
            MERGE (i)-[:INSTANCE_OF]->(sup)
            """)
    driver.close()

if __name__ == "__main__":
    owl_file = sys.argv[1]
    run_reasoner(owl_file, "inferred.owl")
    import_into_neo4j("inferred.owl")
    materialize_rules()
    print("Pipeline complete.")
```

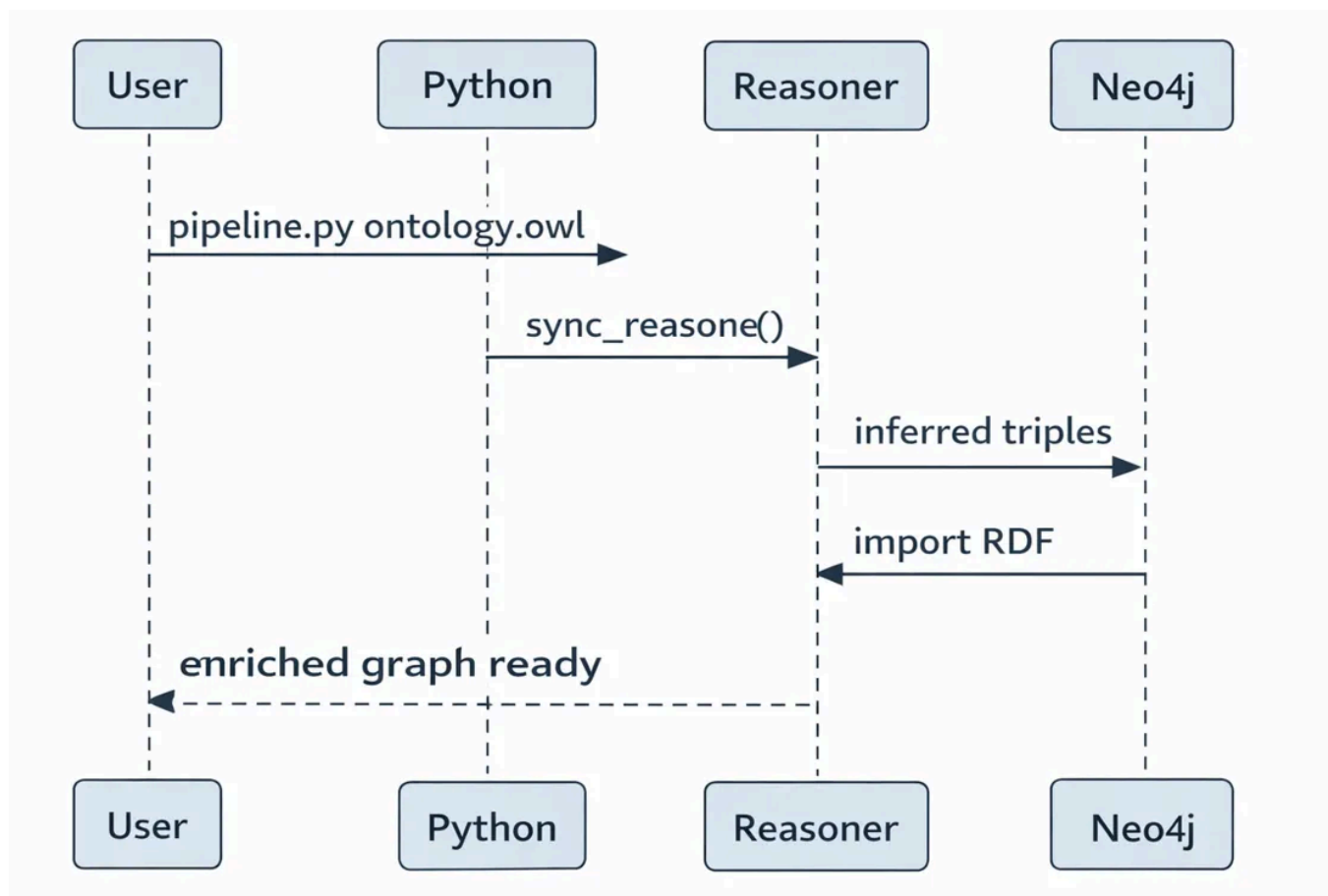
Run:

```
python pipeline.py ontology.owl
```

That single command performs the entire workflow.

What happens internally

The execution flow looks like this:



Your database now physically contains inferred knowledge.

Queries become simpler and faster because no traversal logic is needed.

Optional real-time inference with APOC

If you want near-real-time updates inside Neo4j, you can add procedural rules using APOC:

```
CALL apoc.periodic.repeat(  
  "subclassRule",  
  "MATCH (i)-[:INSTANCE_OF]->(c)-[:SUBCLASS_OF]->(sup)  
    MERGE (i)-[:INSTANCE_OF]->(sup)",  
  5  
);
```

This runs every few seconds and mimics lightweight reasoning.

It's not as expressive as OWL, but useful for simple propagation.

When this architecture shines

This hybrid model works extremely well for enterprise knowledge graphs because it separates concerns cleanly.

Neo4j handles operational workloads at high speed. The reasoner handles semantics periodically or nightly. The graph remains performant, but logically enriched.

It avoids trying to force a graph database to become a theorem prover, which is neither efficient nor elegant.

Final intuition

Think of the stack like this:

- Neo4j stores facts
- The ontology defines meaning

- The reasoner derives new truths
- The pipeline merges everything

Once you internalize that separation, ontology-based reasoning with Neo4j becomes straightforward rather than mysterious.

Ontology

Reasoning

Owlready

Rdfliib

Cypher



Follow

Written by Akash Goyal

1.1K followers · 233 following

Building Agentic AI applications | Generative AI | LLMs | Agents | NLP | Data Scientist | Machine Learning

<https://www.linkedin.com/in/akash-g-7a5224246/>

Responses (2)



Stellars

What are your thoughts?



Kevin Hughes

Feb 11



Very impressive and makes tremendous sense. How do you build in uncertainty or probability of truth? That is, once you make a triplet, you imply truth. But some triplets are speculations or probable relationships. If the system autonomously assumes... [more](#)



1



1 reply

[Reply](#)

Dr Nicolas Figay

Feb 17



OWL inferences are about typed object properties. With LPG, both nodes and edges have identifiers, so they correspond to IRIs and as a consequence classes or individuals. If possible to attach data properties on nodes and edges, LPGs don't propose... [more](#)

[Reply](#)

More from Akash Goyal



Akash Goyal

Building Agentic GraphOS: The 16-Layer Architecture Behind Production-Ready Knowledge Graphs

A Cost-Optimized, Fully Observable GraphRAG Operating System Built for Enterprise Production



Dec 14, 2025

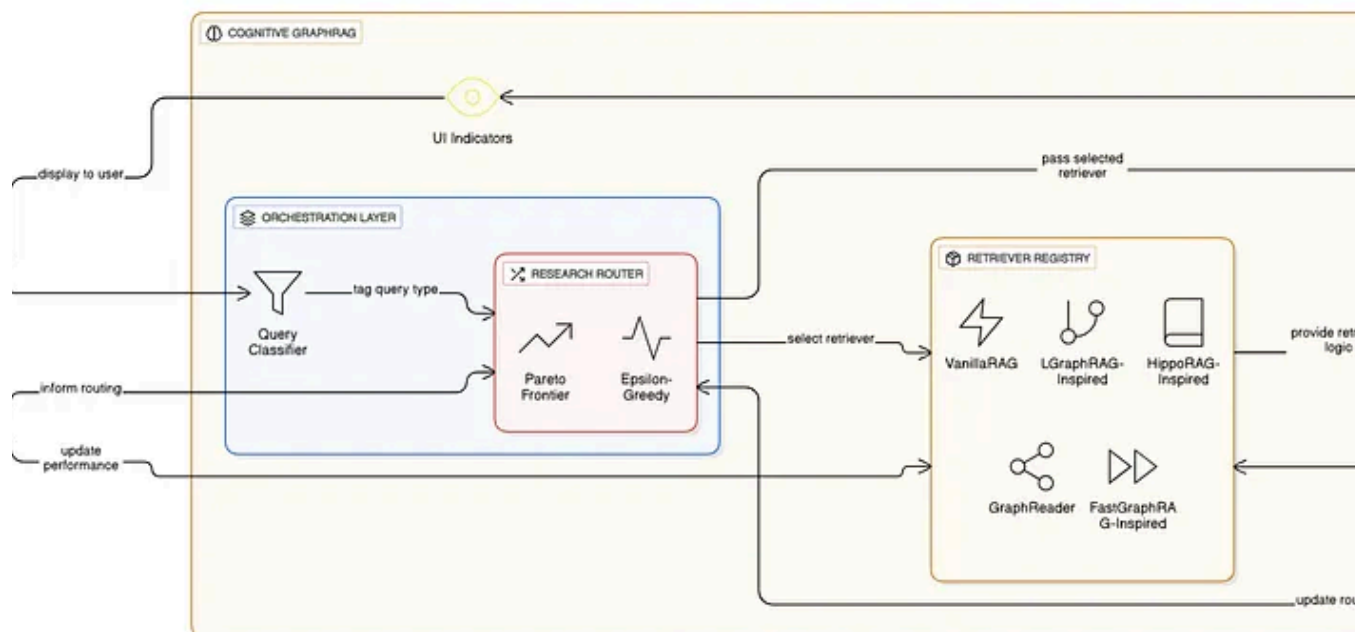


648



7





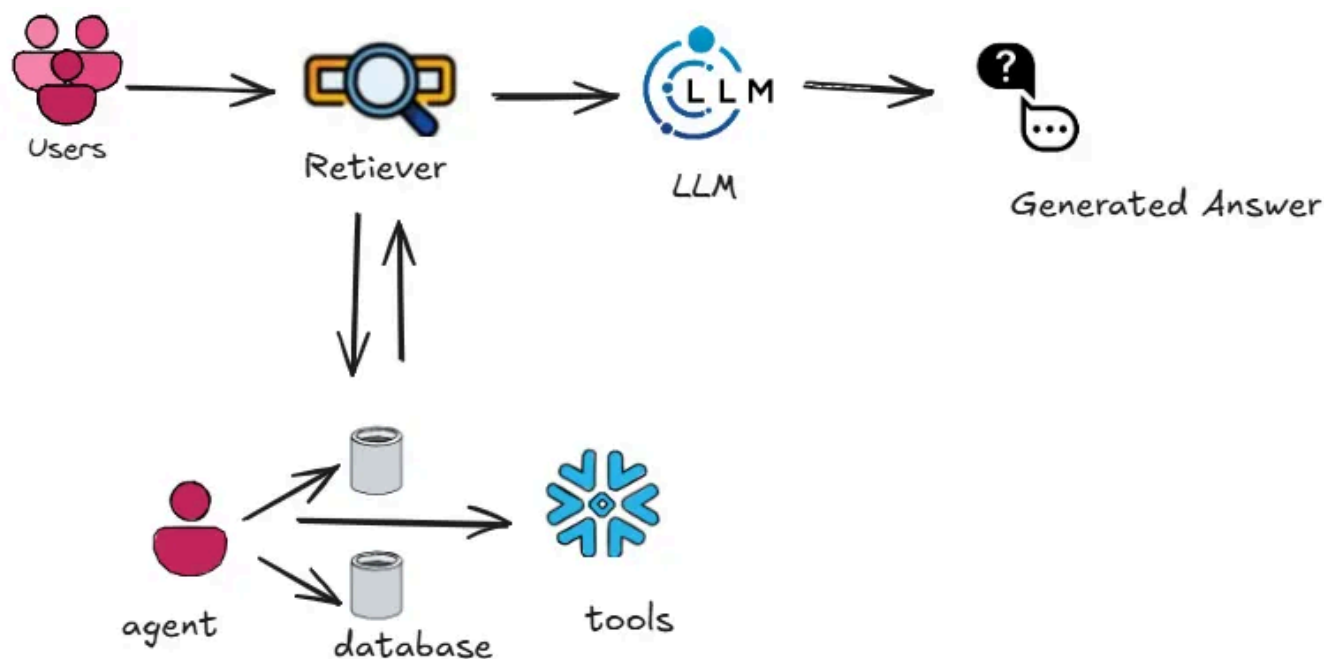
 Akash Goyal

A Self-Optimizing GraphRAG That Selects the Right Retrieval Strategy—Every Time

“How cognitive routing, graph reasoning, and cost-aware decisioning outperform one-size-fits-all GraphRAG.”

★ Dec 13, 2025 🖱 280 💬 3

 ...

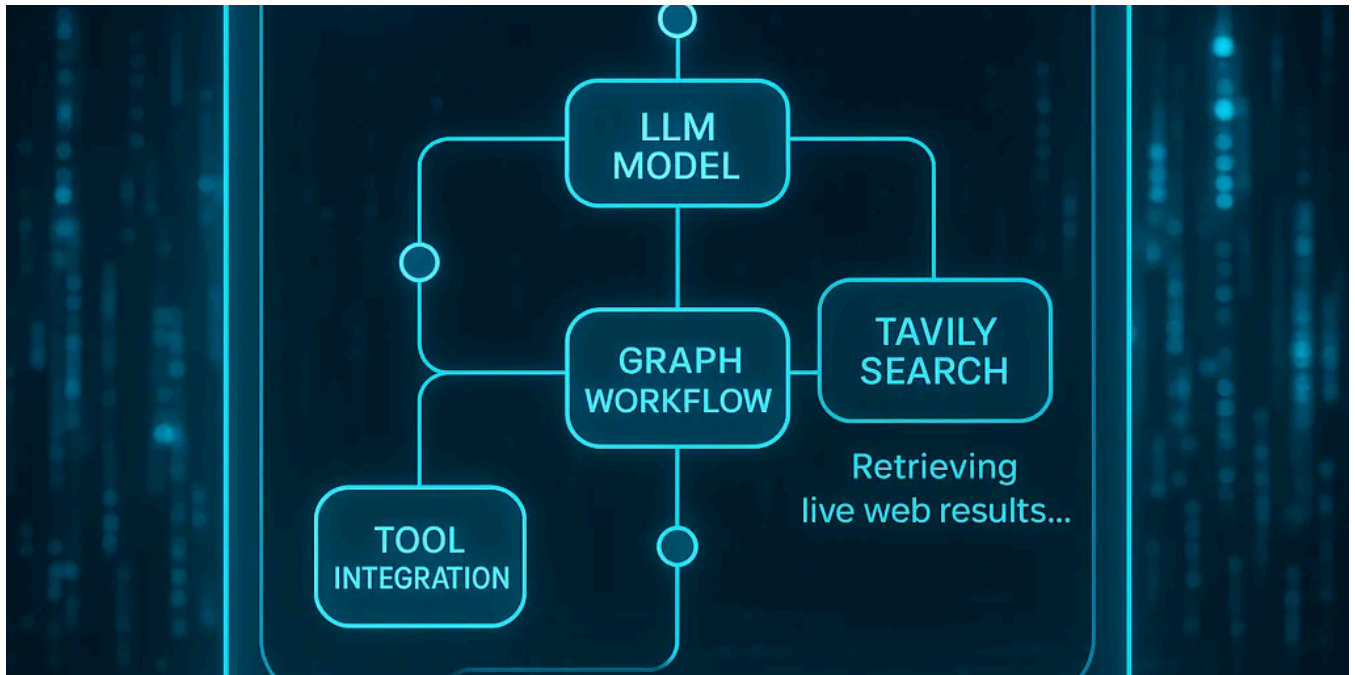


 In Generative AI by Akash Goyal

How to make dumb LLMs smarter with agentic retrieval-augmented generation (RAG)

With LangChain, FAISS and Groq

★ Mar 9, 2025 🖱 116 💬 1

 In Data Science Collective by Akash Goyal

Beyond Basic Bots: A Guide to Streamlit, LangGraph & Tavily on Hugging Face

In this piece, we'll explore a chatbot application built using Streamlit for the user interface and LangGraph for managing the chatbot's...

★ Apr 1, 2025 🖱 38

[See all from Akash Goyal](#)

Recommended from Medium

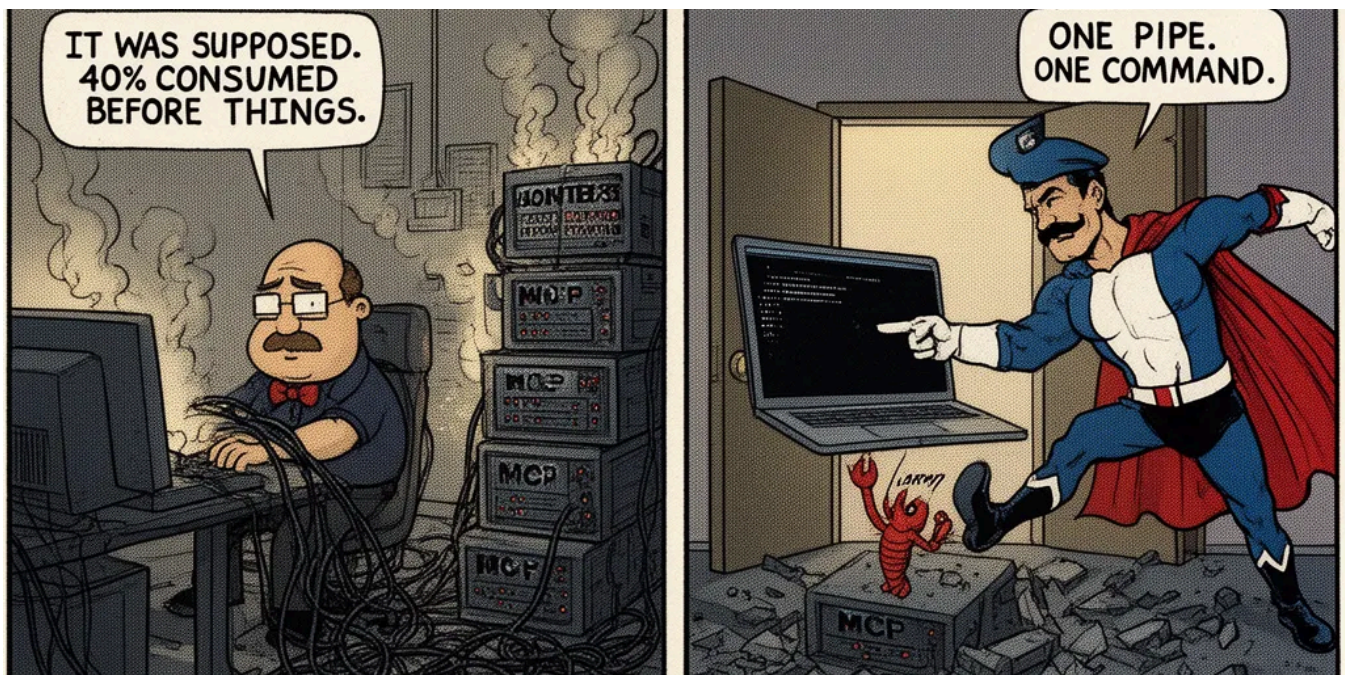


 ADITHYA GIRIDHARAN

Zvec: Alibaba Just Open-Sourced “The SQLite of Vector Databases” — And It’s Blazing Fast

Every decade or so, someone takes a powerful technology that requires dedicated infrastructure and collapses it into a library you can...

Feb 13  37  3



 Phil | Rentier Digital Automation 

Why CLIs Beat MCP for AI Agents — And How to Build Your Own CLI Army.

“mcp were a mistake. bash is better.”

Feb 17 1.4K 33



In ILLUMINATION by Somya Golchha

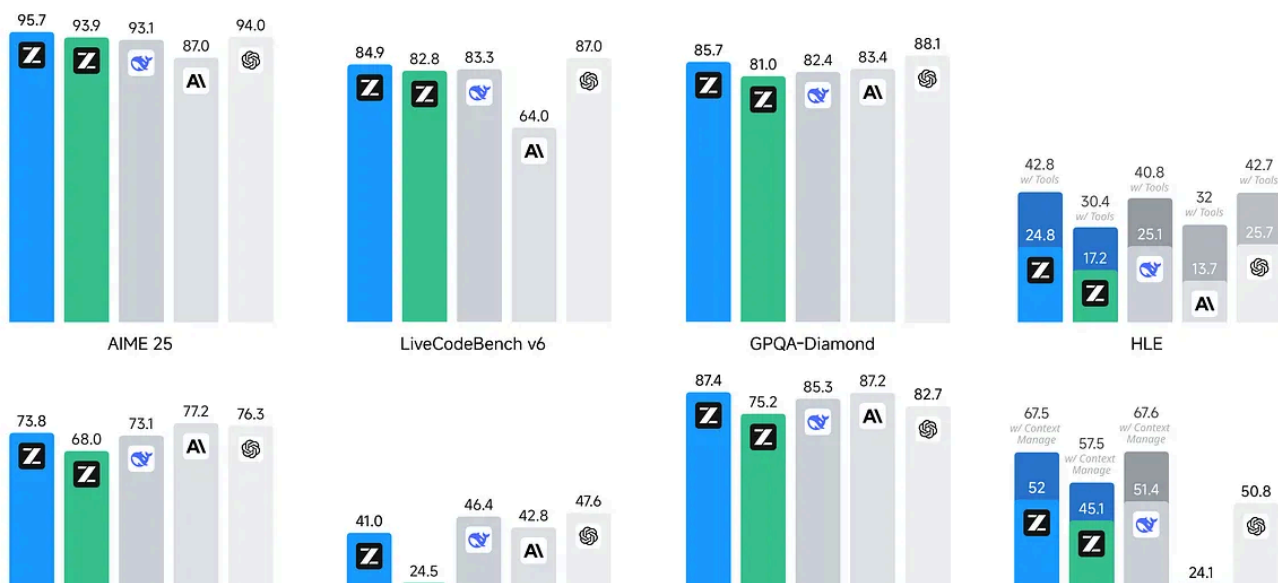
The Microsoft-OpenAI Divorce Is Official: Inside the \$250 Billion Betrayal

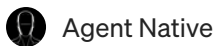
GPT-5 is already being retired, and Satya Nadella just made his move. The “partnership” of the century is entering its terminal phase.

Feb 18 1.3K 30



GLM-4.7 GLM-4.6 DeepSeek-V3.2 Claude Sonnet 4.5 GPT-5.1 (High)





Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >\$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you might be wondering...

★ Jan 20 🖱️ 1.6K 💬 44



 In AI Advances by Harish K

I Tested 5 OCR Models on 6 Real-World Datasets — Here's Which One You Should Actually Use

There's no single “best” OCR engine. Tesseract is fastest, EasyOCR dominates scene text, TrOCR owns handwriting, and DocTR quietly wins...

★ Feb 15 🖱️ 317 💬 7





 In Towards AI by Divy Yadav

A Beginner's Guide to Production-Grade Agentic RAG (6 RAG Patterns Explained with Examples)

Six Agentic RAG patterns explained with real production trade-offs

★ Feb 10 🖱 294 💬 2



See more recommendations